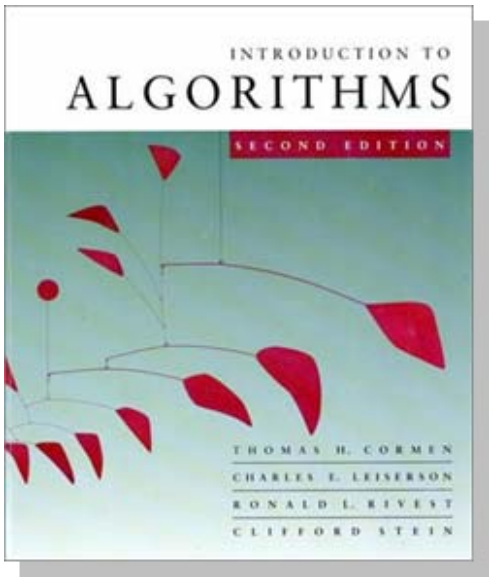
Introduction to Algorithms



LECTURE 5

Sorting Lower Bounds

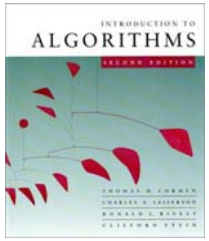
- Decision trees

Linear-Time Sorting

- Counting sort
- Radix sort
- Bucket sort

Appendix: Punched cards

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



How fast can we sort?

All the sorting algorithms we have seen so far are ***comparison sorts***: only use comparisons to determine the relative order of elements.

- *E.g.*, insertion sort, merge sort, quicksort, heapsort.

The best worst-case running time that we've seen for comparison sorting is $O(n \lg n)$.

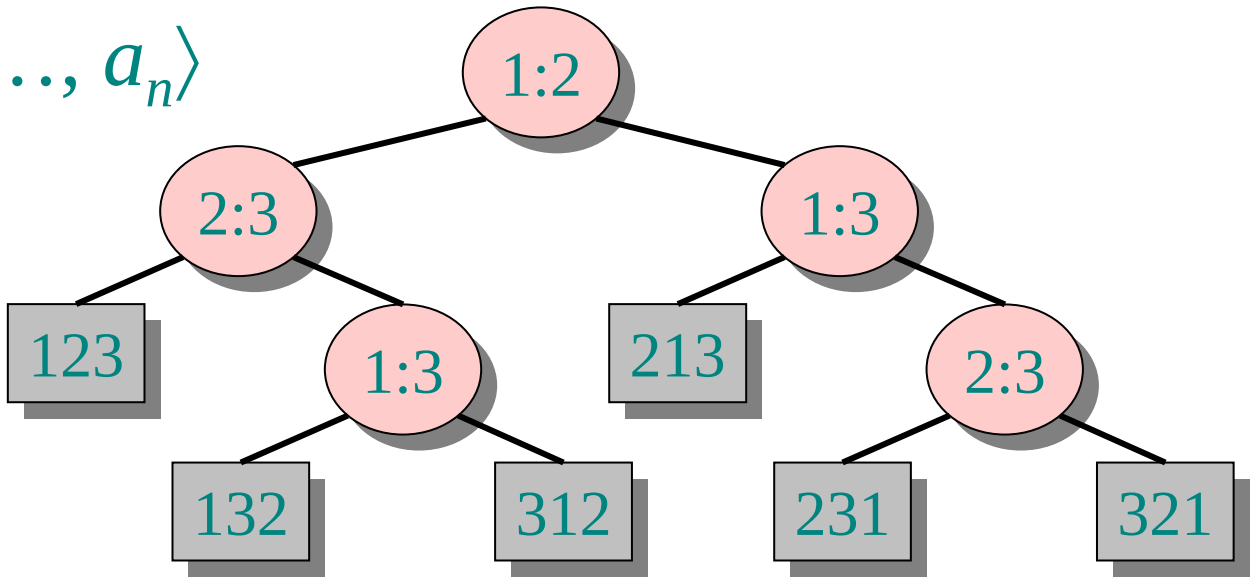
Is $O(n \lg n)$ the best we can do?

Decision trees can help us answer this question.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Decision-tree example

Sort $\langle a_1, a_2, \ldots, a_n \rangle$



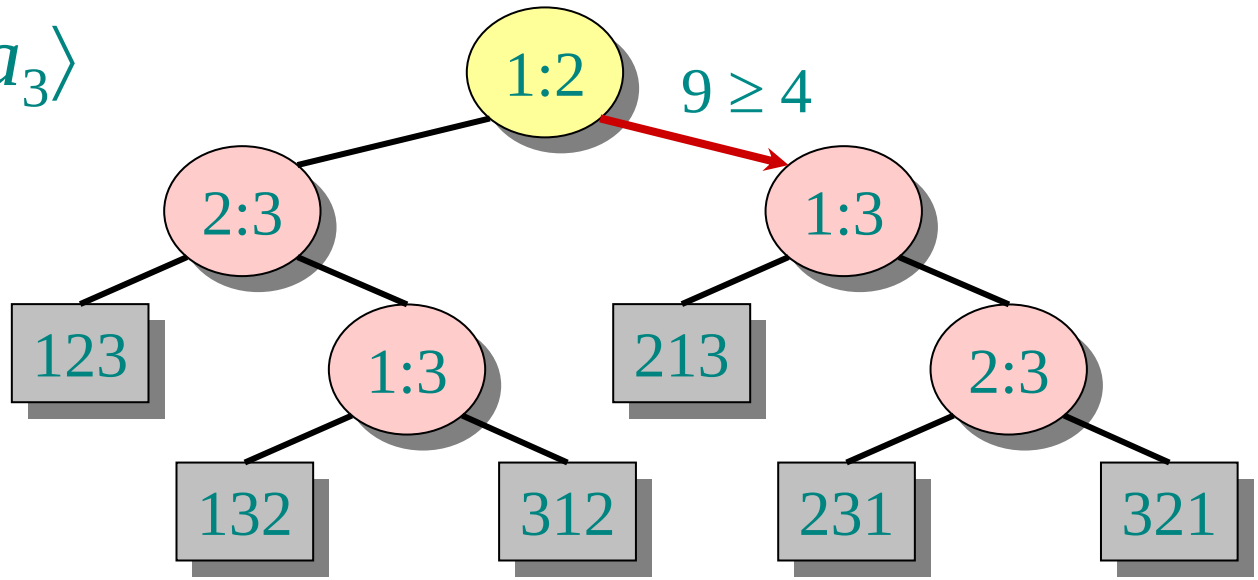
Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \le a_j$.
- The right subtree shows subsequent comparisons if $a_i > a_j$.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$:



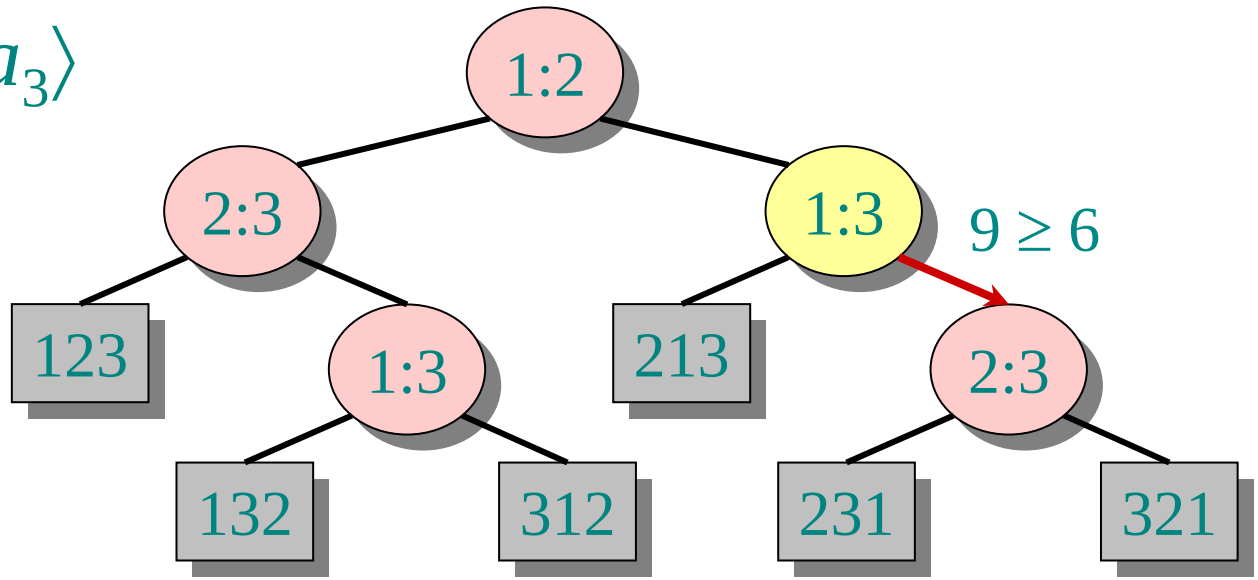
Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \le a_j$.
- The right subtree shows subsequent comparisons if $a_i \ge a_j$.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$:



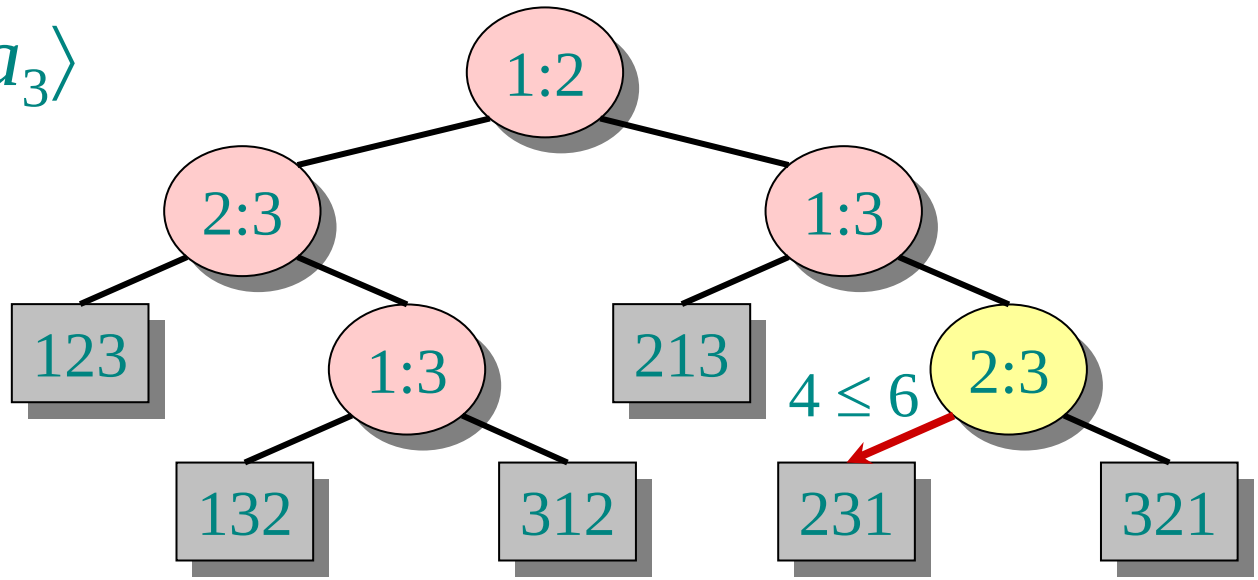
Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \le a_j$.
- The right subtree shows subsequent comparisons if $a_i \ge a_j$.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Decision-tree example

Sort $\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$:



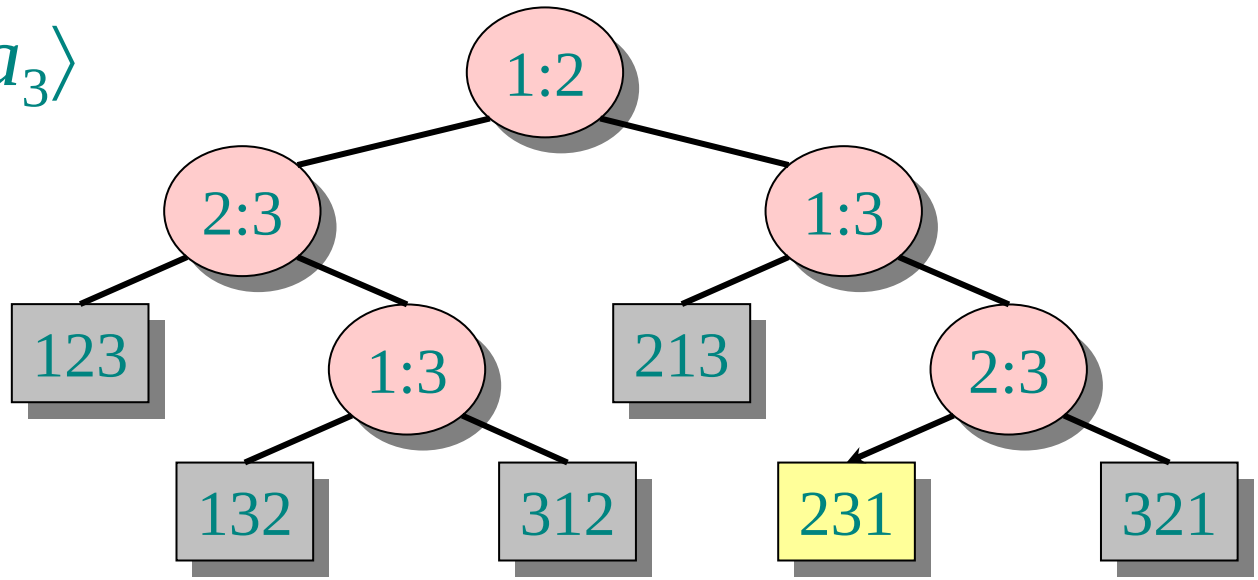
Each internal node is labeled $i{:}j$ for $i, j \in \{1, 2, \ldots, n\}$.

- The left subtree shows subsequent comparisons if $a_i \le a_j$.
- The right subtree shows subsequent comparisons if $a_i \ge a_j$.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Decision-tree example

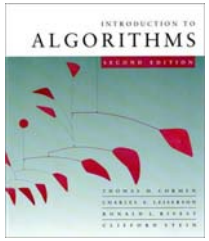
Sort $\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$:



$4 \le 6 \le 9$

Each leaf contains a permutation $\langle \pi(1), \pi(2), \ldots, \pi(n) \rangle$ to indicate that the ordering $a_{\pi(1)} \le a_{\pi(2)} \le \cdots \le a_{\pi(n)}$ has been established.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

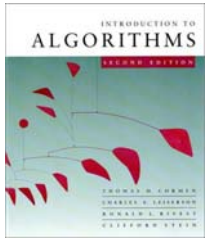


Decision-tree model

A decision tree can model the execution of any comparison sort:

- One tree for each input size n.
- View the algorithm as splitting whenever it compares two elements.
- The tree contains the comparisons along all possible instruction traces.
- The running time of the algorithm = the length of the path taken.
- Worst-case running time = height of tree.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



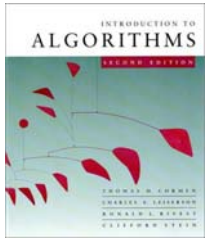
Lower bound for decision-tree sorting

Theorem. Any decision tree that can sort n elements must have height $\Omega(n \lg n)$.

Proof. The tree must contain $\geq n!$ leaves, since there are $n!$ possible permutations. A height-h binary tree has $\leq 2^h$ leaves. Thus, $n! \leq 2^h$.

$$\begin{aligned} \therefore\ h &\geq \lg(n!) && (\lg \text{ is mono. increasing}) \\ &\geq \lg\left((n/e)^n\right) && (\text{Stirling's formula}) \\ &= n \lg n - n \lg e \\ &= \Omega(n \lg n). \end{aligned}$$

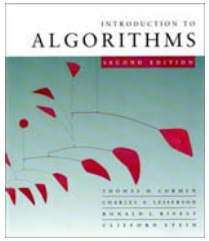
 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Lower bound for comparison sorting

Corollary. Heapsort and merge sort are asymptotically optimal comparison sorting algorithms. □

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

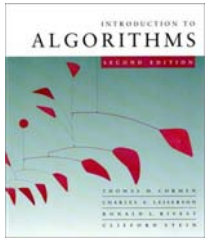


Sorting in linear time

Counting sort: No comparisons between elements.

- ***Input***: $A[1 \ldots n]$, where $A[j] \in \{1, 2, \ldots, k\}$.
- ***Output***: $B[1 \ldots n]$, sorted.
- ***Auxiliary storage***: $C[1 \ldots k]$.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Counting sort

for $i = 1$ **to** k

$\quad C[i] = 0$

for $j = 1$ **to** n

$\quad C[A[j]] = C[A[j]] + 1$ //$C[i] = |\{\text{key} = i\}|$

for $i = 2$ **to** k

$\quad C[i] = C[i] + C[i-1]$ //$C[i] = |\{\text{key} \le i\}|$

for $j=n$ **downto** 1

$\quad B[C[A[j]]] = A[j]$

$\quad C[A[j]] = C[A[j]] - 1$

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Counting-sort example

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:				

B:					

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 1

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	0	0	0	0

B:					

for $i = 1$ **to** k

$C[i] = 0$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 2

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	0	0	0	1

B:					

for $j = 1$ **to** n

$C[A[j]] = C[A[j]] + 1$ //$C[i] = |\{\text{key} = i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 2

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	1	0	0	1

B:					

for $j = 1$ **to** n

$C[A[j]] = C[A[j]] + 1$ //$C[i] = |\{\text{key} = i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 2

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	1	0	1	1

B:					

for j=1 **to** n

$C[A[j]] = C[A[j]] + 1$ //$C[i] = |\{\text{key} = i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 2

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	1	0	1	2

B:					

for $j = 1$ **to** n

$C[A[j]] = C[A[j]] + 1$ //$C[i] = |\{\text{key} = i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 2

	1	2	3	4	5
A:	4	1	3	4	3

	1	2	3	4
C:	1	0	2	2

B:					

for $j = 1$ **to** n

$C[A[j]] = C[A[j]] + 1$ //$C[i] = |\{\text{key} = i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 3

	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	2	2
C':	1	1	2	2

for i=2 **to** k

$C[i] = C[i] + C[i-1]$ //$C[i] = |\{\text{key} \le i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 3

	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	2	2
C':	1	1	3	2

for i=2 **to** k

$C[i] = C[i] + C[i-1]$ //$C[i] = |\{\text{key} \le i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 3

	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	2	2
C':	1	1	3	5

for i=2 **to** k

$C[i] = C[i] + C[i-1]$ //$C[i] = |\{\text{key} \le i\}|$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 4

	1	2	3	4	5
A:	4	1	3	4	3
B:			3		

	1	2	3	4
C:	1	1	3	5
C':	1	1	2	5

for $j=n$ **downto** 1

$B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 4

	1	2	3	4	5
A:	4	1	3	4	3
B:			3		4

	1	2	3	4
C:	1	1	2	5
C':	1	1	2	4

for $j=n$ **downto** 1

$B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 4

	1	2	3	4	5
A:	4	1	3	4	3
B:		3	3		4

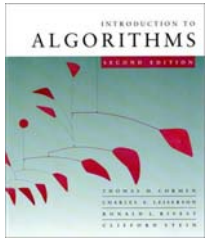
	1	2	3	4
C:	1	1	2	4
C':	1	1	1	4

for $j=n$ **downto** 1

$B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Loop 4

	1	2	3	4	5
A:	4	1	3	4	3
B:	1	3	3		4

	1	2	3	4
C:	1	1	1	4
C':	0	1	1	4

for $j=n$ **downto** 1

$B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Loop 4

	1	2	3	4	5
A:	4	1	3	4	3
B:	1	3	3	4	4

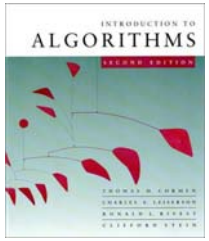
	1	2	3	4
C:	0	1	1	4
C':	0	1	1	3

for $j=n$ **downto** 1

$B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Analysis

$\Theta(k)$ **for** $i=1$ **to** k
$\quad C[i] = 0$

$\Theta(n)$ **for** $j=1$ **to** n
$\quad C[A[j]] = C[A[j]] + 1$

$\Theta(k)$ **for** $i=2$ **to** k
$\quad C[i] = C[i] + C[i-1]$

$\Theta(n)$ **for** $j=n$ **downto** 1
$\quad B[C[A[j]]] = A[j]$
$\quad C[A[j]] = C[A[j]] - 1$

$\Theta(n + k)$

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Running time

If $k = O(n)$, then counting sort takes $\Theta(n)$ time.

- But, sorting takes $\Omega(n \lg n)$ time!
- Where's the fallacy?

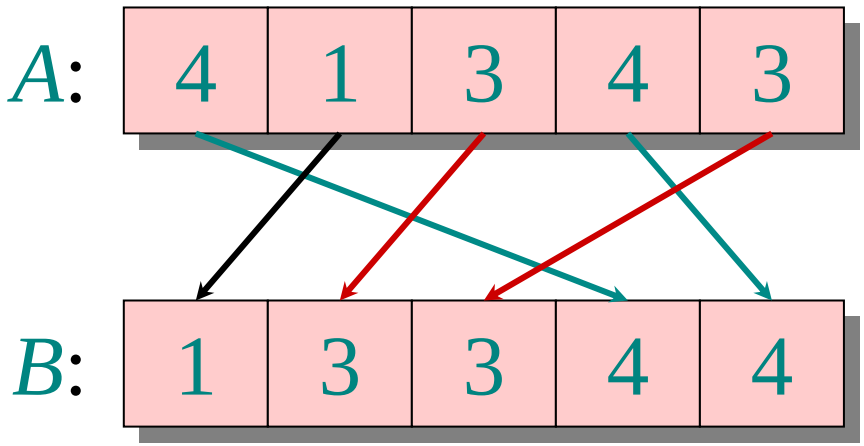
Answer:

- ***Comparison sorting*** takes $\Omega(n \lg n)$ time.
- Counting sort is not a ***comparison sort***.
- In fact, not a single comparison between elements occurs!

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

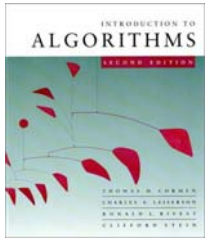
Stable sorting

Counting sort is a ***stable*** sort: it preserves the input order among equal elements.



Exercise: What other sorts have this property?

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Radix sort

- ***Origin***: Herman Hollerith's card-sorting machine for the 1890 U.S. Census. (See Appendix .)
- Digit-by-digit sort.
- Hollerith's original (bad) idea: sort on most-significant digit first.
- Good idea: Sort on ***least-significant digit first*** with auxiliary ***stable*** sort.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Operation of radix sort

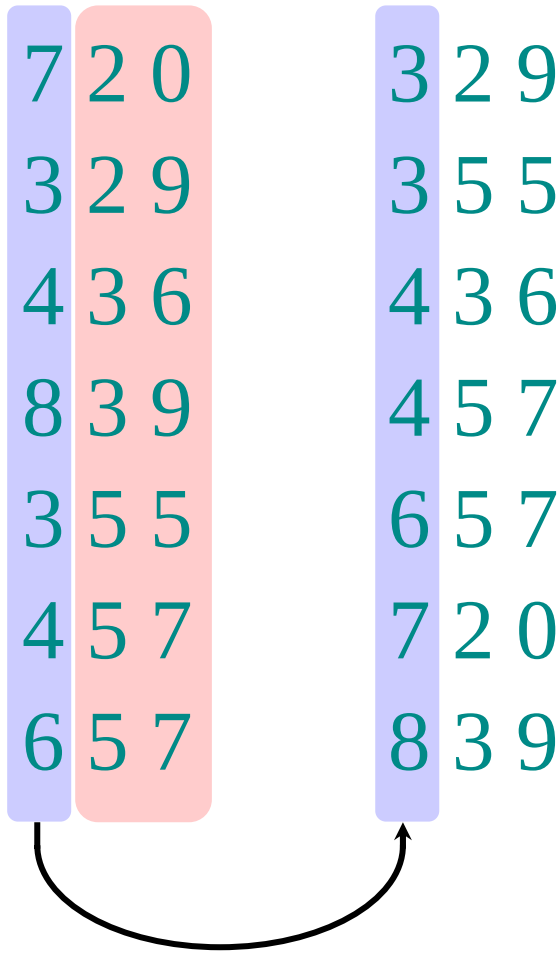
3	2	9	7	2	0	7	2	0	3	2	9
4	5	7	3	5	5	3	2	9	3	5	5
6	5	7	4	3	6	4	3	6	4	3	6
8	3	9	4	5	7	8	3	9	4	5	7
4	3	6	6	5	7	3	5	5	6	5	7
7	2	0	3	2	9	4	5	7	7	2	0
3	5	5	8	3	9	6	5	7	8	3	9

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Correctness of radix sort

Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t

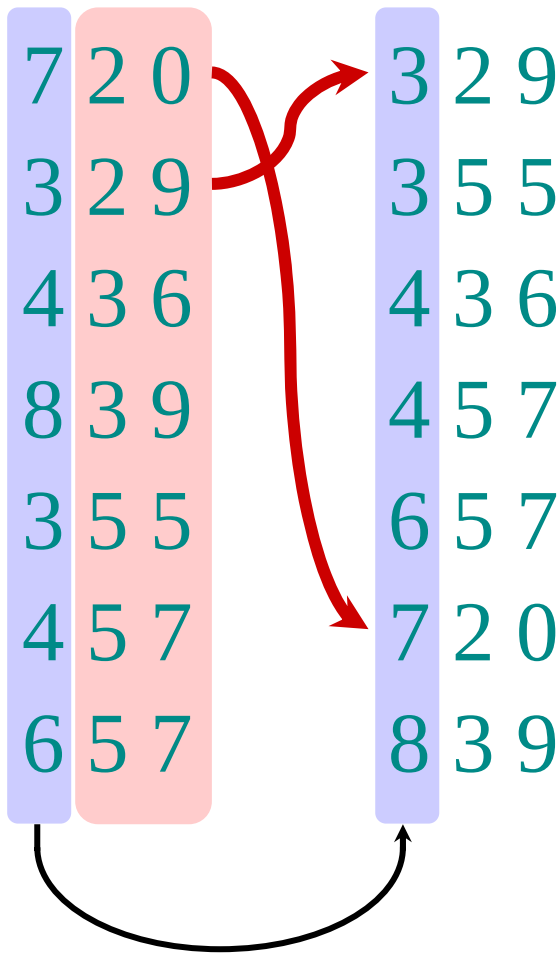


 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Correctness of radix sort

Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
  - Two numbers that differ in digit t are correctly sorted.

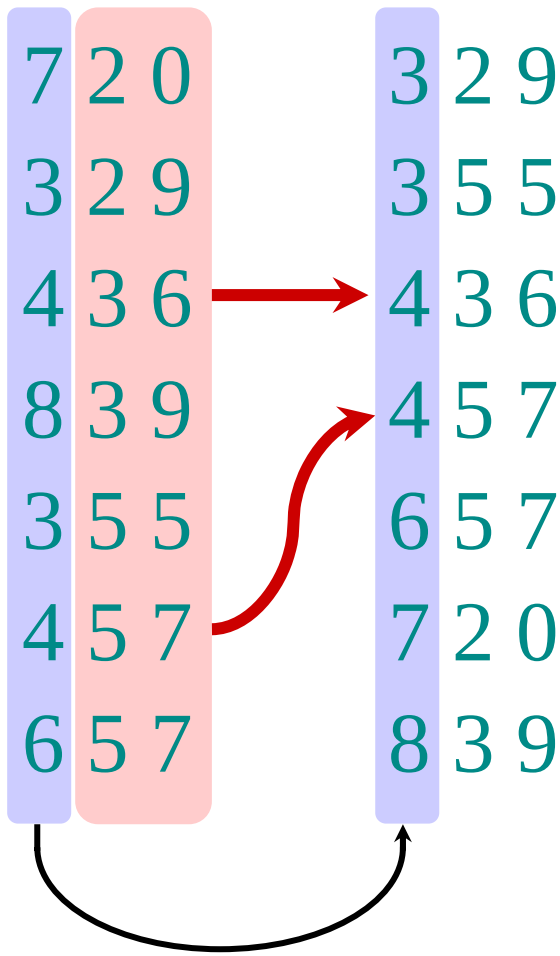


 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

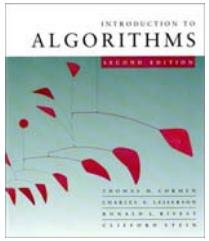
Correctness of radix sort

Induction on digit position

- Assume that the numbers are sorted by their low-order $t - 1$ digits.
- Sort on digit t
  - Two numbers that differ in digit t are correctly sorted.
  - Two numbers equal in digit t are put in the same order as the input $\Rightarrow$ correct order.



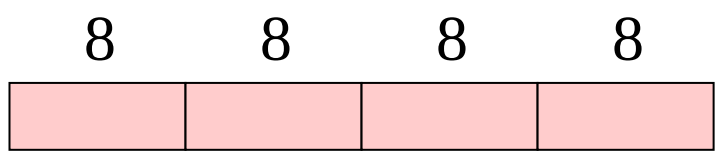
 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Analysis of radix sort

- Assume counting sort is the auxiliary stable sort.
- Sort n computer words of b bits each.
- Each word can be viewed as having b/r base-2^r digits.

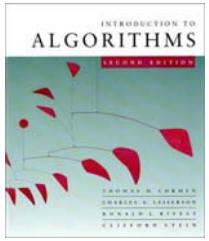
Example: 32-bit word



$r = 8 \Rightarrow b/r = 4$ passes of counting sort on base-2^8 digits; or $r = 16 \Rightarrow b/r = 2$ passes of counting sort on base-2^{16} digits.

How many passes should we make?

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Analysis (continued)

Recall: Counting sort takes $\Theta(n + k)$ time to sort n numbers in the range from 0 to $k - 1$.

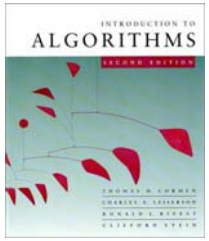
If each b-bit word is broken into r-bit pieces, each pass of counting sort takes $\Theta(n + 2^r)$ time. Since there are b/r passes, we have

$$T(n,b) = \Theta\left(\frac{b}{r}\left(n + 2^r\right)\right).$$

Choose r to minimize $T(n, b)$:

- Increasing r means fewer passes, but as $r \gg \lg n$, the time grows exponentially.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Choosing *r*

$$T(n,b) = \Theta\left(\frac{b}{r}\left(n + 2^r\right)\right)$$

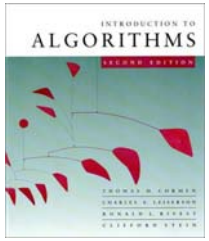
Minimize $T(n, b)$ by differentiating and setting to 0.

Or, just observe that we don't want $2^r \gg n$, and there's no harm asymptotically in choosing r as large as possible subject to this constraint.

Choosing $r = \lg n$ implies $T(n, b) = \Theta(bn/\lg n)$.

- For numbers in the range from 0 to $n^d - 1$, we have $b = d \lg n \Rightarrow$ radix sort runs in $\Theta(d\,n)$ time.

 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Conclusions

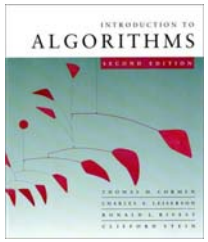
In practice, radix sort is fast for large inputs, as well as simple to code and maintain.

Example (32-bit numbers):

- At most 3 passes when sorting ≥ 2000 numbers.
- Merge sort and quicksort do at least $\lceil \lg 2000 \rceil = 11$ passes.

Downside: Unlike quicksort, radix sort displays little locality of reference, and thus a well-tuned quicksort fares better on modern processors, which feature steep memory hierarchies.

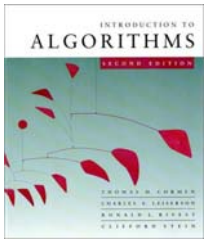
 Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Bucket sort

- ***Idea***: Partitioning an array into a number of buckets, each bucket is then sorted individually
- Bucket sort works as follows:

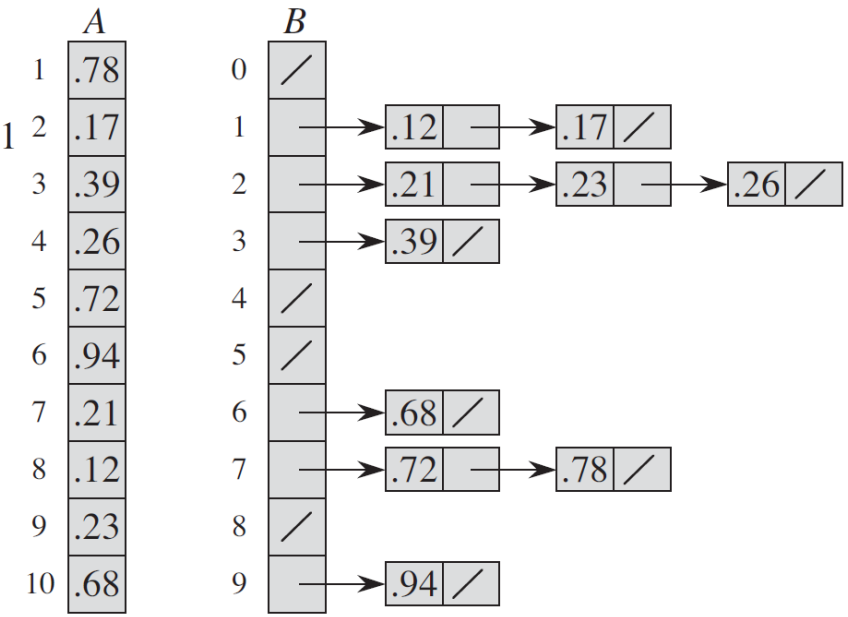
1. Set up an array of initially empty "buckets".
2. **Scatter**: Go over the original array, putting each object in its bucket.
3. Sort each non-empty bucket
4. **Gather**: Visit the buckets in order and put all elements back into the original array.



Bucket sort

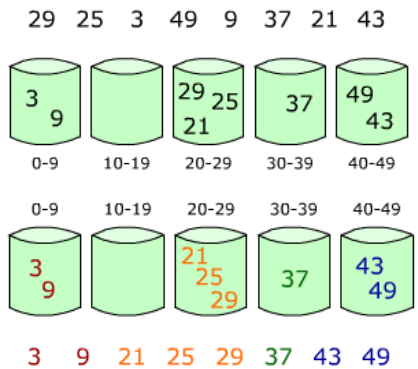
Assume

$0 \le A[i] < 1$



BUCKET-SORT(A)

```
let B[0..n − 1] be a new array
n = A.length
for i = 0 to n − 1
    make B[i] an empty list
for i = 1 to n
    insert A[i] into list B[⌊nA[i]⌋]
for i = 0 to n − 1
    sort list B[i] with insertion sort
concatenate the lists B[0], B[1], ..., B[n − 1] together in order
```



Analysis of bucket sort:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

$$\mathrm{E}\left[n_i^2\right] = 2 - 1/n$$

$$\mathrm{E}[T(n)] = \mathrm{E}\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right]$$

$$= \Theta(n) + n \cdot O(2 - 1/n) = \Theta(n)$$